High Level Design Document

Introduction

This High Level Design (HLD) document outlines the architecture and core components for **DataSnap - Visual Data Insights**, a web-based educational tool. DataSnap enables users to upload CSV files and instantly view simple visualizations (bar and pie charts), introducing students to backend data processing and frontend chart rendering using modern web technologies.

---

1. System Architecture Overview

Architecture Description:
DataSnap follows a client-server model. The frontend (web client) interacts with a backend API (Spring Boot) for file upload and data processing. The backend parses CSV files and returns processed data for visualization. The frontend renders charts using the processed data.

Main System Components:

Component	Role/Responsibility
Web Client (UI)	User interface for file upload and chart visualization
Backend API	Receives files, processes CSV data, exposes endpoints
Data Processor	Parses CSV, aggregates data for charts
Chart Renderer	Renders bar/pie charts in the browser

---

2. Component Interactions

Step	Interaction Description
1	User uploads CSV via Web Client
2	Web Client sends file to Backend API (HTTP POST)
3	Backend API invokes Data Processor to parse and aggregate CSV data
4	Backend API returns processed data (JSON) to Web Client
5	Web Client uses Chart Renderer to display bar/pie charts based on received data

---

3. Data Flow Overview

Source	Data	Destination	Purpose
User	CSV file	Web Client	Initiate data upload
Web Client	CSV file (multipart)	Backend API	Data processing request

Backend API	Processed JSON data	Web Client	Visualization input
Web Client	Chart config/data	Chart Renderer	Render visualizations

---

4. Technology Stack

Layer	Technology/Frameworks
Frontend	HTML5, CSS3, JavaScript, Chart.js (or similar)
Backend	Java, Spring Boot
Data Parsing	OpenCSV (or similar Java CSV library)
Communication	RESTful HTTP (JSON)
Deployment	Standalone Spring Boot app, browser-based client

---

5. Scalability & Reliability

- **Scalability:** Designed for single-user or classroom use; can be extended for multi-user scenarios by containerizing the backend or deploying behind a load balancer.
- **Reliability:** Stateless backend ensures easy recovery; input validation and error handling for file uploads.
- **Security:** Basic file type/size validation; restricts processing to CSV files; no persistent storage of user data.

End of Document